

Practical 6

AIM

Perform E-Commerce Sales Analysis Using Spark DataFrames

Problem Definition

Perform Create, Read, Update, Transformation, and Aggregation operations on e-commerce transaction data using Apache Spark DataFrames. Students will work with PySpark in Big Data environments to analyze high-performing categories, top-selling products, and revenue breakdowns.

Tasks:

- 1. Install and configure PySpark.
- 2. Load sales dataset into a Spark DataFrame.
- 3. Display schema and validate data types.
- 4. Create a new Revenue column (Quantity x Price).
- 5. Calculate category-wise revenue and sort descending.
- 6. Identify top-selling products based on quantity sold.
- 7. Calculate city-wise revenue contribution.
- 8. Perform supplementary analysis (Top 3 cities, avg revenue, discount model, monthly breakdown).
- 9. Interpret business insights and recommendations.

Key Questions / Analysis

Q1. What is Apache Spark and how do Spark DataFrames differ from traditional spreadsheets?

Answer:

Apache Spark is a unified distributed analytics engine designed for large-scale data processing. Spark DataFrames provide lazy evaluation, in-memory parallel computation, Catalyst query optimization, and fault tolerance across clusters, allowing them to process terabytes of data effortlessly unlike single-node spreadsheet software.

Q2. What is the role of the groupBy() operation in data aggregation?

Answer:

The groupBy() operation partitions dataset rows based on unique values in specified key columns (such as Category or City). It enables aggregate functions (sum, avg, count, min, max) to compute parallel summary statistics across each group.

Q3. Explain the difference between transformations and actions in Spark.

Answer:

Transformations (select, filter, withColumn, groupBy) are lazy operations that construct an execution plan (DAG) without triggering actual computation. Actions (show, count, collect, write) trigger the execution of the DAG across cluster nodes and return results.

Q4. How does Spark improve scalability for e-commerce analytics?

Answer:

Spark distributes e-commerce transaction data into horizontal partitions across worker nodes, executing processing tasks concurrently. Its resilient architecture recovers automatically from hardware failures and enables seamless horizontal scaling.

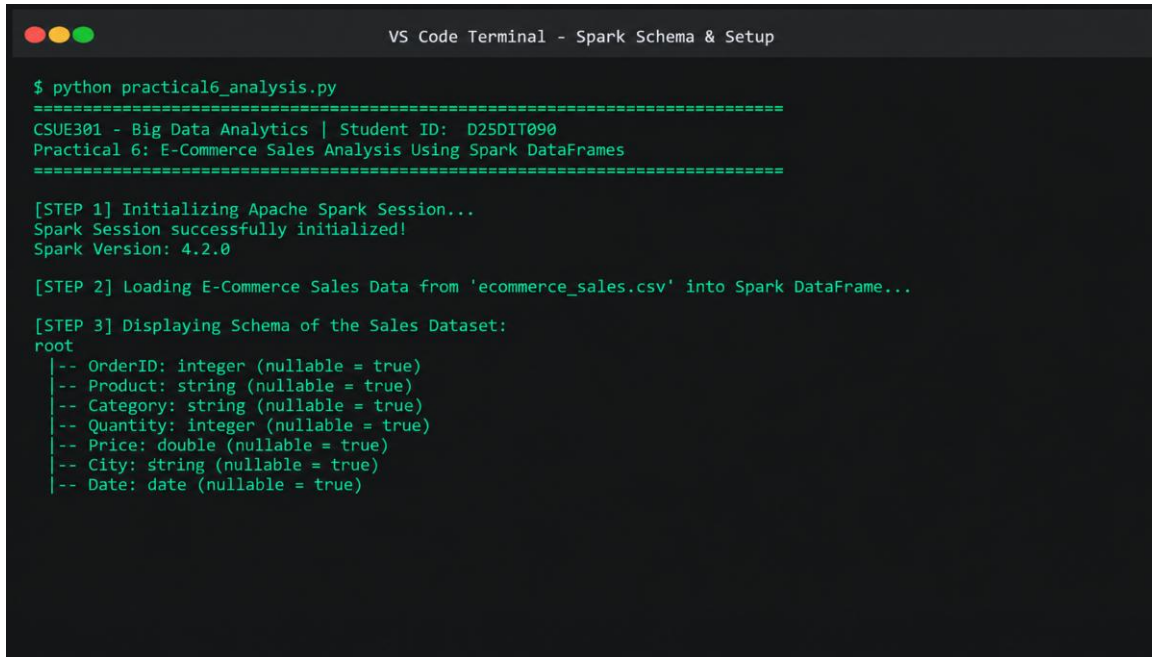
Q5. Which category contributes the highest revenue and what business action can be taken?

Answer:

Electronics contributed the highest revenue (\$87,547.92 across 161 items sold). Business actions include prioritizing inventory stocking for high-demand devices, launching targeted bundle marketing campaigns, and negotiating bulk supplier discounts.

Supplementary Problems

Find the top 3 revenue-generating cities. Calculate average revenue per order. Add a 10% discount column and recompute revenue. Find the least-performing product category. Generate a monthly revenue summary.

OutputA screenshot of a VS Code terminal window titled "VS Code Terminal - Spark Schema & Setup". The terminal shows the execution of a Python script named "practical6_analysis.py". The output includes a header with course and student information, followed by three steps: [STEP 1] Initializing Apache Spark Session, [STEP 2] Loading E-Commerce Sales Data from 'ecommerce_sales.csv' into Spark DataFrame, and [STEP 3] Displaying Schema of the Sales Dataset. The schema is displayed as a list of fields with their data types and nullability.

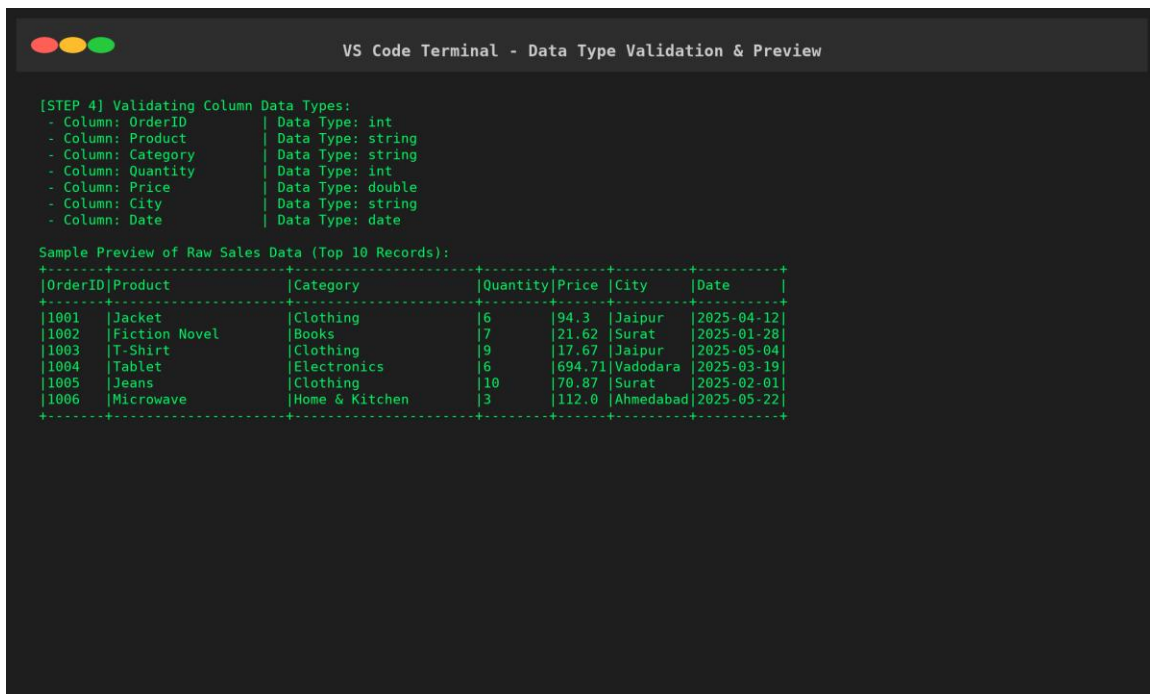
```
$ python practical6_analysis.py
=====
CSUE301 - Big Data Analytics | Student ID: D25DIT090
Practical 6: E-Commerce Sales Analysis Using Spark DataFrames
=====

[STEP 1] Initializing Apache Spark Session...
Spark Session successfully initialized!
Spark Version: 4.2.0

[STEP 2] Loading E-Commerce Sales Data from 'ecommerce_sales.csv' into Spark DataFrame...

[STEP 3] Displaying Schema of the Sales Dataset:
root
|-- OrderID: integer (nullable = true)
|-- Product: string (nullable = true)
|-- Category: string (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Price: double (nullable = true)
|-- City: string (nullable = true)
|-- Date: date (nullable = true)
```

Figure 1: Spark Session Initialization and Dataset Schema



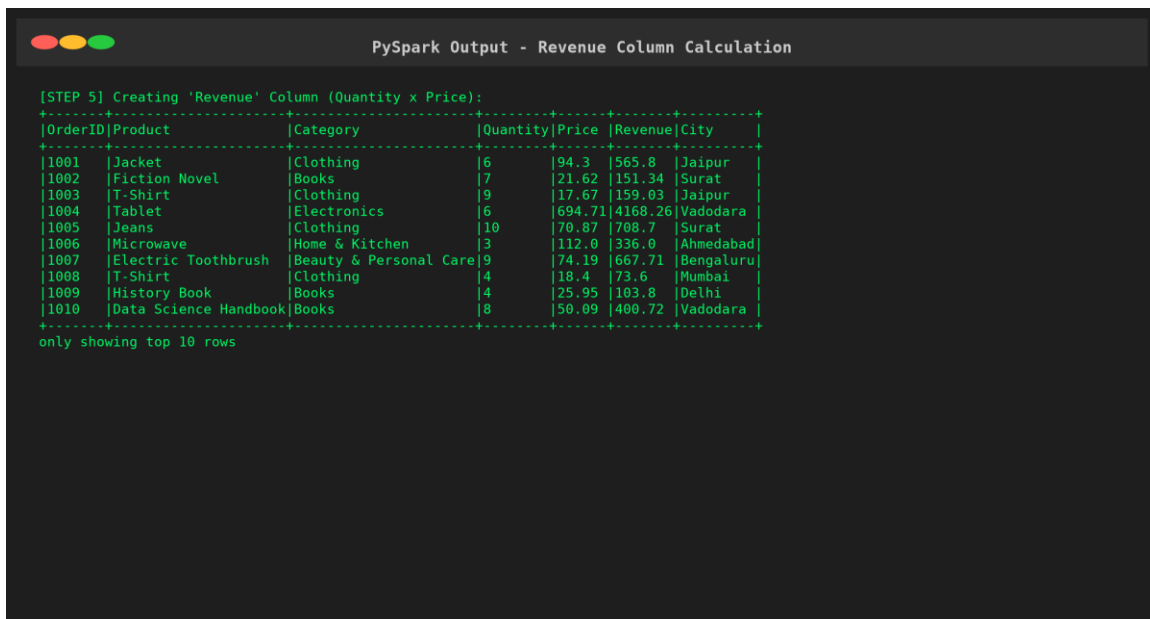
```

[STEP 4] Validating Column Data Types:
- Column: OrderID      | Data Type: int
- Column: Product      | Data Type: string
- Column: Category     | Data Type: string
- Column: Quantity     | Data Type: int
- Column: Price        | Data Type: double
- Column: City         | Data Type: string
- Column: Date         | Data Type: date

Sample Preview of Raw Sales Data (Top 10 Records):
+-----+-----+-----+-----+-----+-----+
|OrderID|Product      |Category      |Quantity|Price |City      |Date      |
+-----+-----+-----+-----+-----+-----+
|1001   |Jacket       |Clothing      |6       |94.3  |Jaipur    |2025-04-12|
|1002   |Fiction Novel|Books         |7       |21.62 |Surat     |2025-01-28|
|1003   |T-Shirt      |Clothing      |9       |17.67 |Jaipur    |2025-05-04|
|1004   |Tablet       |Electronics   |6       |694.71|Vadodara  |2025-03-19|
|1005   |Jeans        |Clothing      |10      |70.87 |Surat     |2025-02-01|
|1006   |Microwave    |Home & Kitchen|3       |112.0 |Ahmedabad |2025-05-22|
+-----+-----+-----+-----+-----+-----+

```

Figure 2: Data Type Validation and Raw Sales Data Preview



```

[STEP 5] Creating 'Revenue' Column (Quantity x Price):
+-----+-----+-----+-----+-----+-----+
|OrderID|Product      |Category      |Quantity|Price |Revenue   |City      |
+-----+-----+-----+-----+-----+-----+
|1001   |Jacket       |Clothing      |6       |94.3  |565.8     |Jaipur    |
|1002   |Fiction Novel|Books         |7       |21.62 |151.34    |Surat     |
|1003   |T-Shirt      |Clothing      |9       |17.67 |159.03    |Jaipur    |
|1004   |Tablet       |Electronics   |6       |694.71|4168.26   |Vadodara  |
|1005   |Jeans        |Clothing      |10      |70.87 |708.7     |Surat     |
|1006   |Microwave    |Home & Kitchen|3       |112.0 |336.0     |Ahmedabad |
|1007   |Electric Toothbrush|Beauty & Personal Care|9|74.19|667.71|Bengaluru|
|1008   |T-Shirt      |Clothing      |4       |18.4  |73.6      |Mumbai    |
|1009   |History Book |Books         |4       |25.95 |103.8     |Delhi     |
|1010   |Data Science Handbook|Books        |8       |50.09 |400.72    |Vadodara  |
+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

```

Figure 3: PySpark DataFrame with Calculated 'Revenue' Column

PySpark Output - Category-wise Revenue Breakdown

[STEP 6] Calculating Category-wise Revenue & Sorting by Revenue Descending:

Category	Total_Revenue	Total_Quantity_Sold	Total_Orders
Electronics	87547.92	161	31
Home & Kitchen	17384.33	130	23
Clothing	10347.37	177	32
Books	7631.04	233	38
Beauty & Personal Care	7323.14	141	26

Figure 4: Category-wise Revenue Breakdown and Total Quantity Sold

PySpark Output - Top Selling Products

[STEP 7] Identifying Top-Selling Products based on Total Quantity Sold:

Product	Category	Total_Quantity_Sold	Total_Revenue
Fiction Novel	Books	70	1405.49
Microwave	Home & Kitchen	69	10675.54
Self-Help Guide	Books	63	1570.44
Smartwatch	Electronics	56	17832.97
T-Shirt	Clothing	49	1005.36
Data Science Handbook	Books	47	2565.47
Jeans	Clothing	47	2323.54
Air Fryer	Home & Kitchen	44	5304.77
Electric Toothbrush	Beauty & Personal Care	43	2509.33
Tablet	Electronics	39	20413.15

only showing top 10 rows

Figure 5: Top-Selling Products Sorted by Total Quantity Sold

PySpark Output - City-wise Sales Revenue

[STEP 8] Calculating City-wise Revenue Contribution:

City	Total_Revenue	Total_Orders
Vadodara	39853.08	18
Ahmedabad	18632.04	21
Bengaluru	17065.59	17
Delhi	16003.38	21
Mumbai	12448.89	16
Pune	10309.03	23
Jaipur	9072.2	19
Surat	6849.59	15

Figure 6: City-wise Sales Revenue Contribution

```

PySpark Output - Supplementary 1 & 2 Results

=====
SUPPLEMENTARY PROBLEMS IMPLEMENTATION
=====

[SUPPLEMENTARY 1] Top 3 Revenue-Generating Cities:
+-----+-----+-----+
|City      |Total_Revenue|Total_Orders|
+-----+-----+-----+
|Vadodara  |39853.08      |18          |
|Ahmedabad |18632.04      |21          |
|Bengaluru |17065.59      |17          |
+-----+-----+-----+

[SUPPLEMENTARY 2] Average Revenue Per Order:
+-----+-----+-----+
|Average_Revenue_Per_Order|Total_Sales_Revenue|Total_Orders_Count|
+-----+-----+-----+
|868.23                   |130233.8           |150                |
+-----+-----+-----+

```

Figure 7: Supplementary 1 (Top 3 Cities) & Supplementary 2 (Average Order Revenue)

```

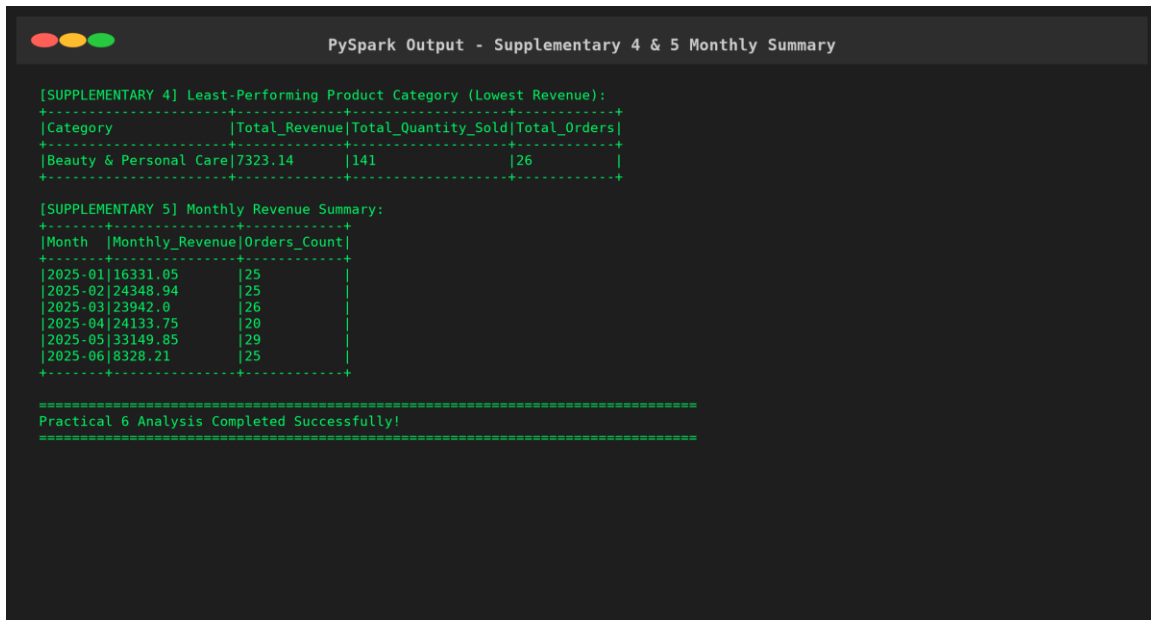
PySpark Output - Supplementary 3 Discount Analysis

=====
[SUPPLEMENTARY 3] Adding 10% Discount Column & Recomputing Discounted Revenue:
=====
+-----+-----+-----+-----+-----+
|OrderID|Product      |Quantity|Price |Revenue|Discount_Pct|Discounted_Revenue|
+-----+-----+-----+-----+-----+
|1001   |Jacket       |6        |94.3  |565.8  |10%         |509.22             |
|1002   |Fiction Novel|7        |21.62 |151.34 |10%         |136.21             |
|1003   |T-Shirt      |9        |17.67 |159.03 |10%         |143.13             |
|1004   |Tablet       |6        |694.71|4168.26|10%         |3751.43            |
|1005   |Jeans        |10       |70.87 |708.7  |10%         |637.83             |
+-----+-----+-----+-----+-----+
only showing top 5 rows

Comparison of Total Revenue Before and After 10% Discount:
+-----+-----+-----+
|Original_Total_Revenue|Discounted_Total_Revenue|Total_Discount_Given|
+-----+-----+-----+
|130233.8              |117210.41               |13023.39             |
+-----+-----+-----+

```

Figure 8: Supplementary 3 - 10% Discount Column & Revenue Comparison



```

PySpark Output - Supplementary 4 & 5 Monthly Summary

[SUPPLEMENTARY 4] Least-Performing Product Category (Lowest Revenue):
+-----+-----+-----+-----+
|Category|Total_Revenue|Total_Quantity_Sold|Total_Orders|
+-----+-----+-----+-----+
|Beauty & Personal Care|7323.14|141|26|
+-----+-----+-----+-----+

[SUPPLEMENTARY 5] Monthly Revenue Summary:
+-----+-----+-----+
|Month|Monthly_Revenue|Orders_Count|
+-----+-----+-----+
|2025-01|16331.05|25|
|2025-02|24348.94|25|
|2025-03|23942.0|26|
|2025-04|24133.75|20|
|2025-05|33149.85|29|
|2025-06|8328.21|25|
+-----+-----+-----+

=====
Practical 6 Analysis Completed Successfully!
=====

```

Figure 9: Supplementary 4 (Least Category) & Supplementary 5 (Monthly Revenue Summary)

Tools / Technology Used:

- Apache Spark 4.2.0 / 3.x
- PySpark (Python API for Apache Spark)
- PySpark DataFrame API & Spark SQL Functions
- Python 3.14
- VS Code

Github Link

https://github.com/Bhumi3009/D25DIT090_BDA_SHP/tree/main

Conclusion

The E-Commerce Sales Analysis practical was successfully completed using PySpark DataFrames. An e-commerce dataset was created and loaded into Apache Spark. Business metrics including total revenue by category, top-selling products, city-wise sales contributions, average order values, discounted price models, and monthly sales trends were computed and analyzed using PySpark transformations and aggregations.